Overview of Statistical Learning and Modeling – 36-600

Project 3: Classification
Due Friday, November 21, 11:59pm

Trishla Nair

The goal of this project is to use classification to see if we can distinguish wine by its charcteristics. The dataset includes 12 features including the target and 6497 observations. The features examine the composition of the wines.

```
library(tidyverse)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   4.0.0     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.1
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ggplot2)

df<- read.csv("C:/Users/Trish/Documents/CMU/CMU-3660/Projects/wineQuality.csv")
df$label <- ifelse(df$label == "GOOD", 1, 0)
df$label <- as.factor(df$label)
y <-df$label
print(summary(df))
```

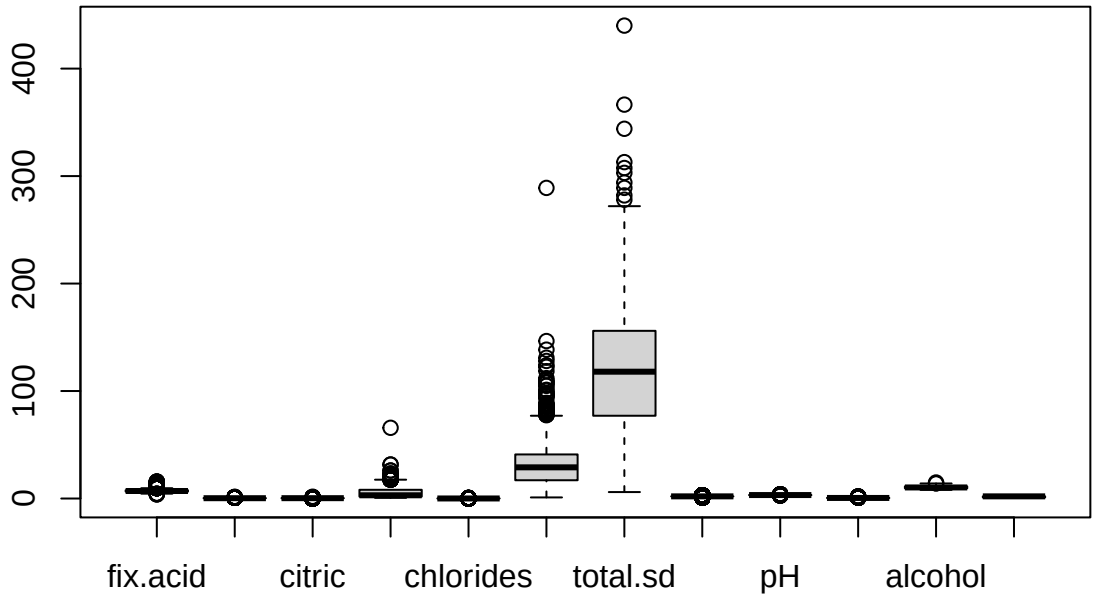
```
##     fix.acid         vol.acid          citric           sugar
##  Min.   : 3.800   Min.   :0.0800   Min.   :0.0000   Min.   : 0.600
##  1st Qu.: 6.400   1st Qu.:0.2300   1st Qu.:0.2500   1st Qu.: 1.800
##  Median : 7.000   Median :0.2900   Median :0.3100   Median : 3.000
##  Mean   : 7.215   Mean   :0.3397   Mean   :0.3186   Mean   : 5.443
##  3rd Qu.: 7.700   3rd Qu.:0.4000   3rd Qu.:0.3900   3rd Qu.: 8.100
##  Max.   :15.900   Max.   :1.5800   Max.   :1.6600   Max.   :65.800
##    chlorides          free.sd          total.sd       density
##  Min.   :0.00900   Min.   :  1.00   Min.   :  6.0   Min.   :1.00
##  1st Qu.:0.03800   1st Qu.: 17.00   1st Qu.: 77.0   1st Qu.:2.00
##  Median :0.04700   Median : 29.00   Median :118.0   Median :2.00
##  Mean   :0.05603   Mean   : 30.53   Mean   :115.7   Mean   :1.97
##  3rd Qu.:0.06500   3rd Qu.: 41.00   3rd Qu.:156.0   3rd Qu.:2.00
```

```
##  Max.   :0.61100   Max.   :289.00   Max.   :440.0   Max.   :3.00
##        pH          sulphates         alcohol      label
##  Min.   :2.720   Min.   :0.2200   Min.   : 8.00   0:2384
##  1st Qu.:3.110   1st Qu.:0.4300   1st Qu.: 9.50   1:4113
##  Median :3.210   Median :0.5100   Median :10.30
##  Mean   :3.219   Mean   :0.5313   Mean   :10.49
##  3rd Qu.:3.320   3rd Qu.:0.6000   3rd Qu.:11.30
##  Max.   :4.010   Max.   :2.0000   Max.   :14.90
```

```
print(dim(df))
```

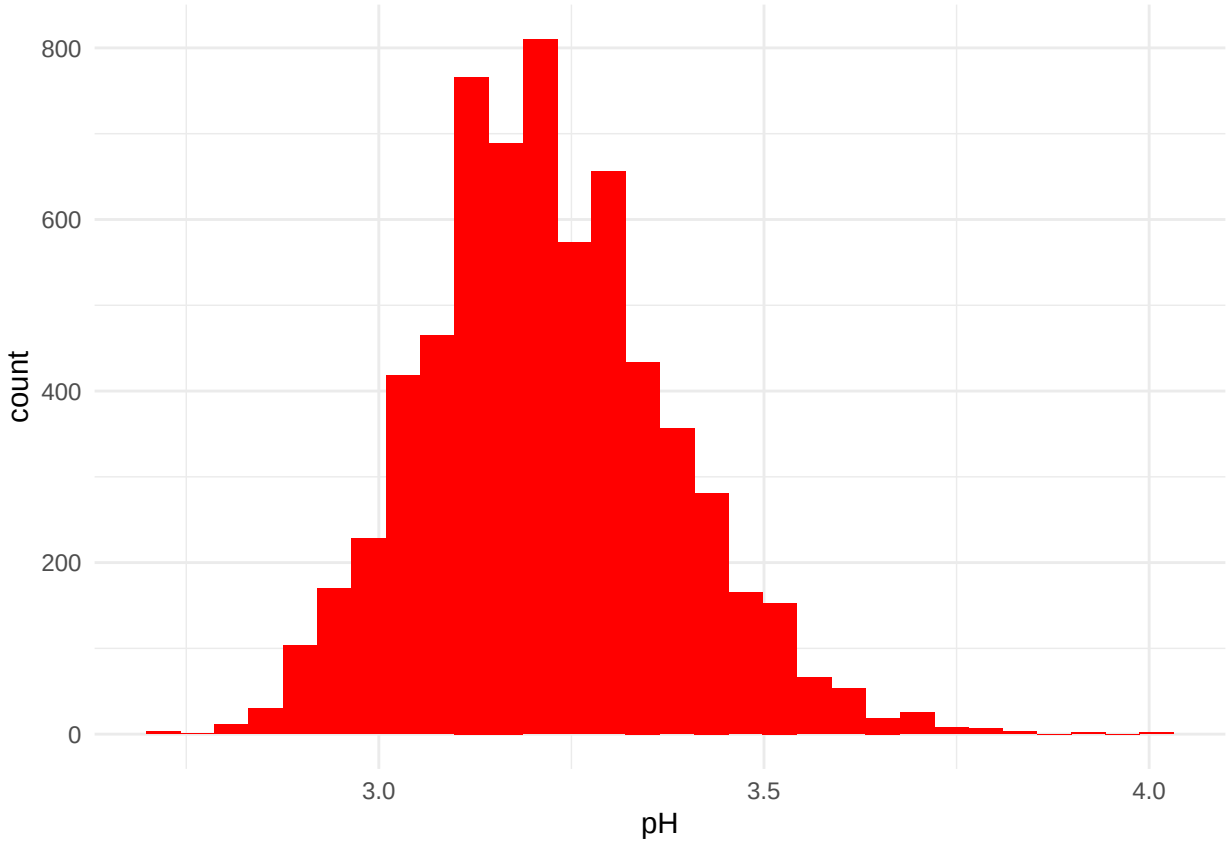
```
## [1] 6497   12
```

```
boxplot(df)
```

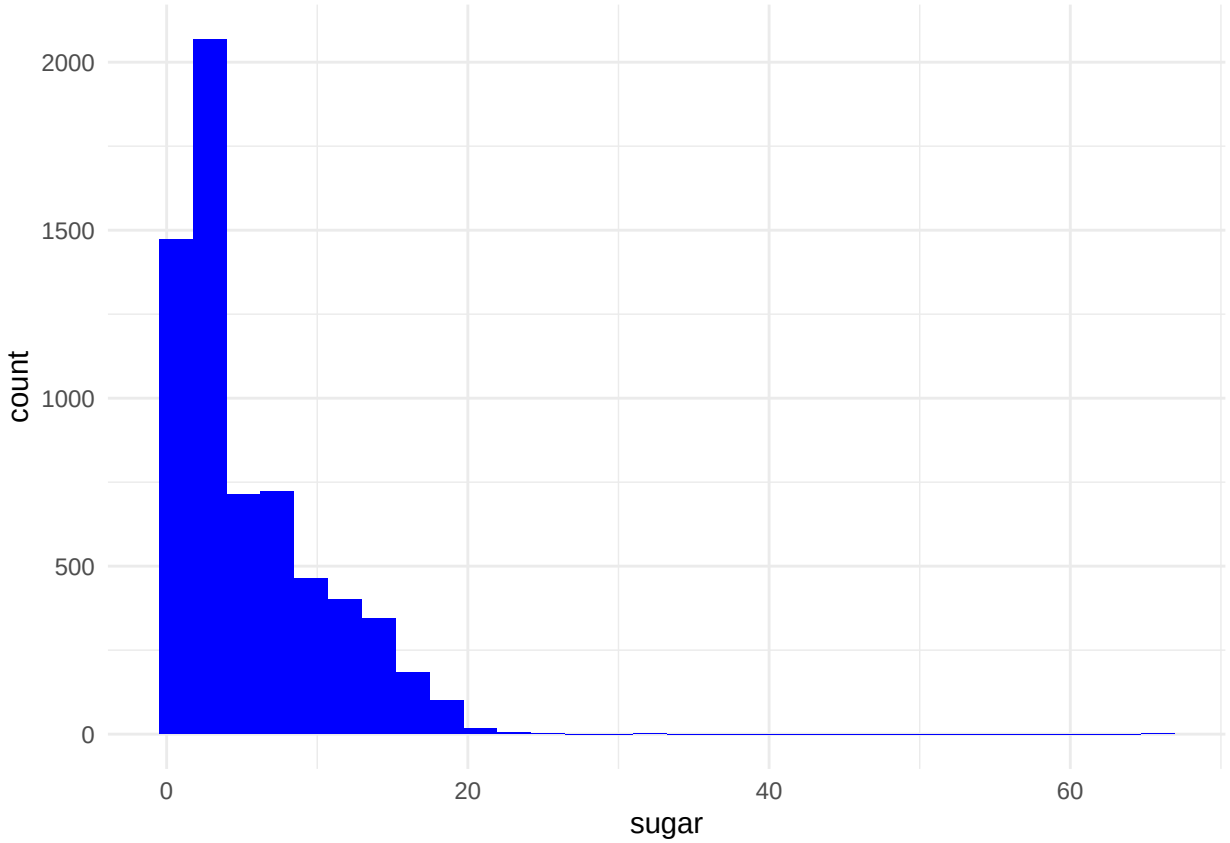


based on the box plot, we can see there is general little variability except for the total.sd and free.sd features. As we are looking into wine I also looked into the disrtibutions of pH, sugar, and alcohol levels.

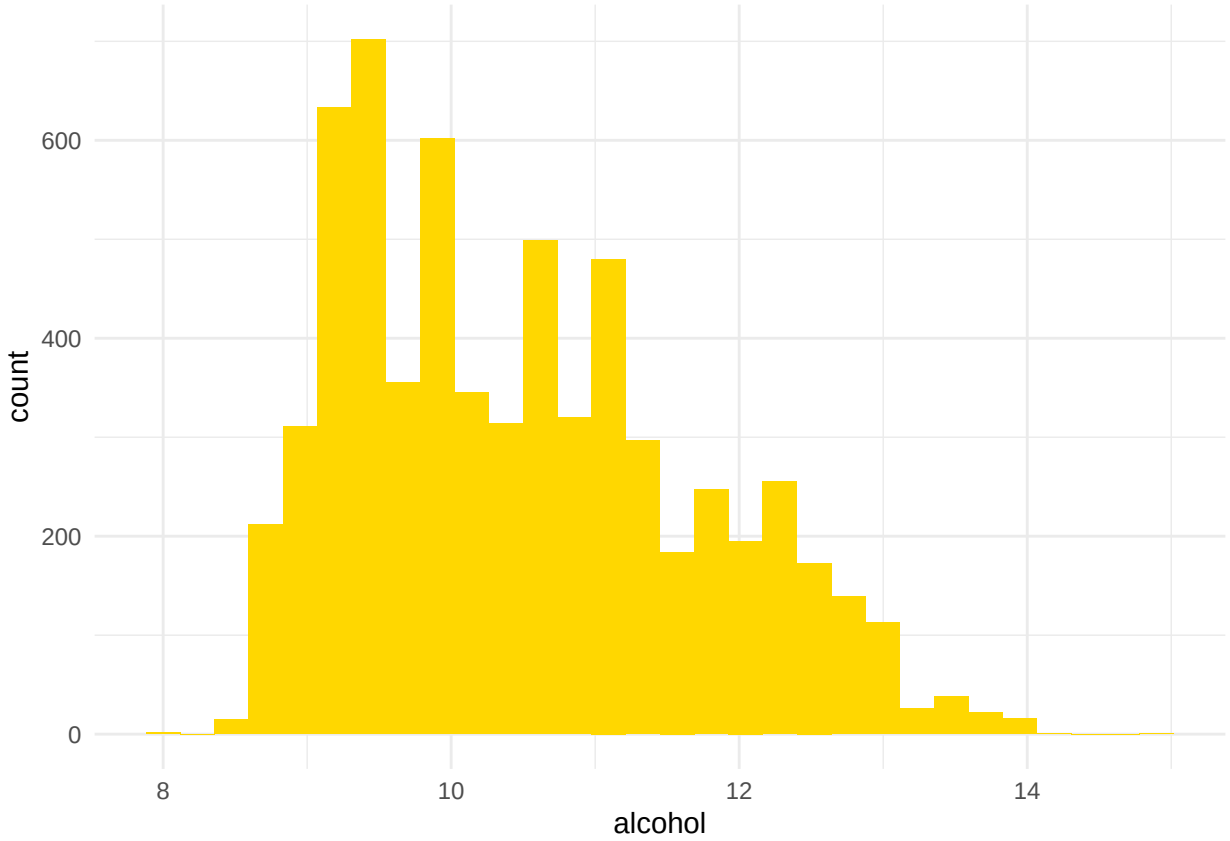
```
ggplot(df, aes(pH)) + geom_histogram(bins = 30, fill = "red") + theme_minimal()
```




```
ggplot(df, aes(sugar)) + geom_histogram(bins = 30, fill = "blue") + theme_minimal()
```




```
ggplot(df, aes(alcohol)) + geom_histogram(bins = 30, fill = "gold") + theme_minimal()
```



After examining the distributions of these variables, they are generally unimodal, but visibly skewed, so I would need to standardized the data.

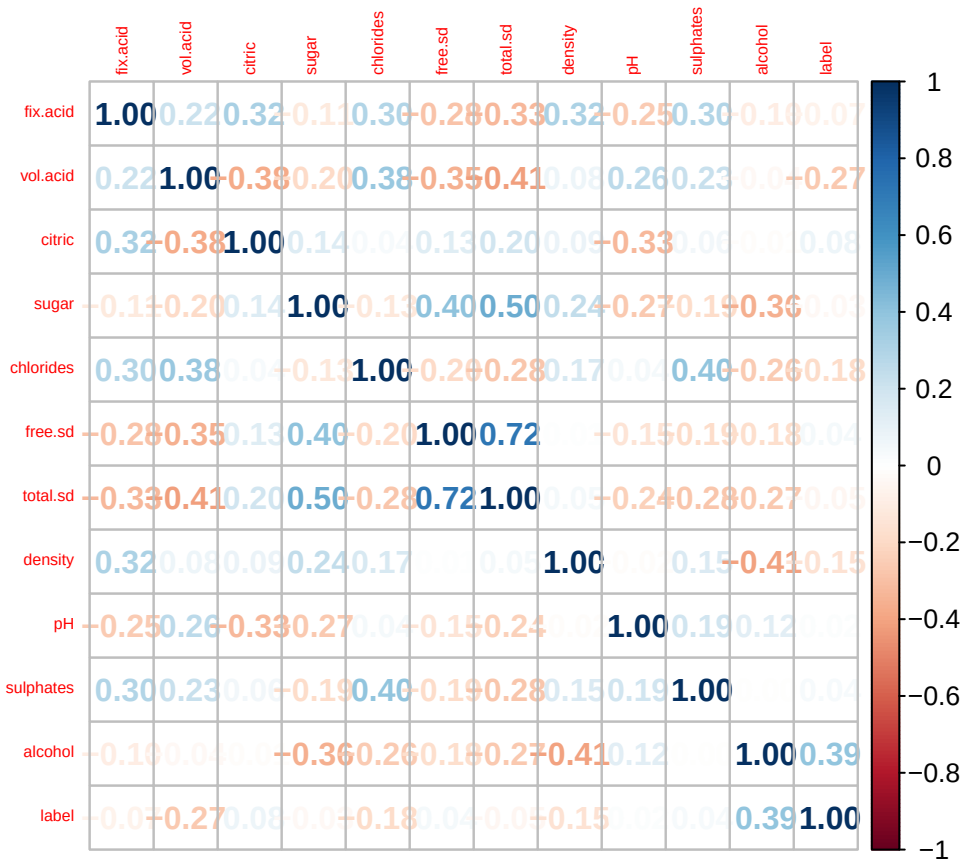
After that, I generated a correlation matrix:

```
library(corrplot)
```

```
## Warning: package 'corrplot' was built under R version 4.4.3
```

```
## corrplot 0.95 loaded
```

```
df$label <- as.numeric(as.character(df$label))
corrplot(cor(df), method = "number",tl.cex =0.5)
```



Based on the matrix, the terms do not seem to be highly correlated withh the target variable. The highest correlation however seems to be between sugar and total. sd and total.sd and free.sd.

I then ran a few logistic regression models with and without minteraction terms to see which would perform better based on the AUC.

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.4.3
```

```
## Loading required package: lattice
```

```
##
## Attaching package: 'caret'
```

```
## The following object is masked from 'package:purrr':
##
##     lift
```

```
set.seed(42)
n <- nrow(df)
train_idx <- sample(1:n, 0.7 * n)
train_set <- df[train_idx,]
test_set  <- df[-train_idx, ]
```

```r
predictor_cols <- setdiff(names(train_set), "label")

# Standardize predictors
preProc <- preProcess(train_set[, predictor_cols], method = c("center", "scale"))
train_scaled <- train_set
train_scaled[, predictor_cols] <- predict(preProc, train_set[, predictor_cols])
test_scaled <- test_set
test_scaled[, predictor_cols] <- predict(preProc, test_set[, predictor_cols])
```

```r
library(pROC)
```

```
## Warning: package 'pROC' was built under R version 4.4.3
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```r
logit_model <- glm(label ~ ., data = train_scaled, family = "binomial")
summary(logit_model)
```

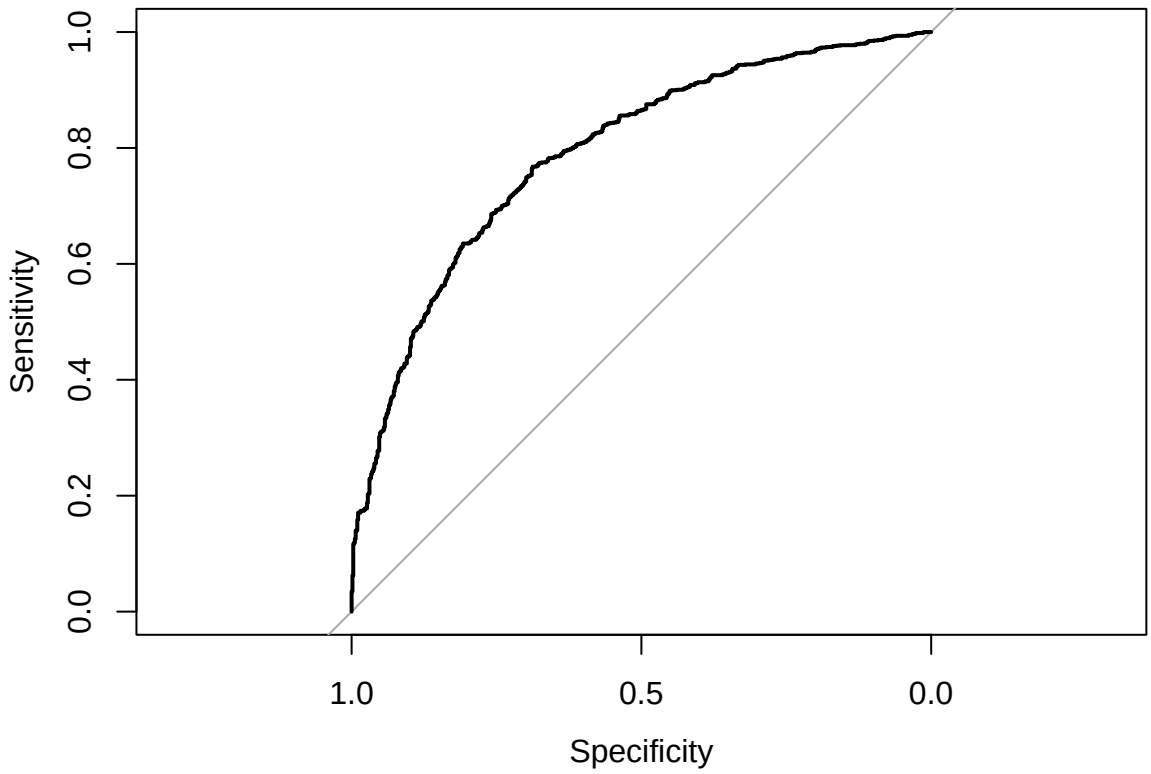
```
##
## Call:
## glm(formula = label ~ ., family = "binomial", data = train_scaled)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.75205    0.03866  19.453  < 2e-16 ***
## fix.acid     0.04786    0.05006   0.956    0.339
## vol.acid    -0.79741    0.05056 -15.772  < 2e-16 ***
## citric      -0.08274    0.04400  -1.881    0.060 .
## sugar        0.30282    0.04448   6.808 9.91e-12 ***
## chlorides   -0.02537    0.04319  -0.587    0.557
## free.sd      0.30628    0.05396   5.676 1.38e-08 ***
## total.sd    -0.44254    0.05917  -7.480 7.45e-14 ***
## density     -0.07060    0.05254  -1.344    0.179
## pH           0.07403    0.04367   1.695    0.090 .
## sulphates    0.26478    0.04430   5.977 2.27e-09 ***
## alcohol      1.11572    0.05188  21.505  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 5979.2  on 4546  degrees of freedom
## Residual deviance: 4642.1  on 4535  degrees of freedom
## AIC: 4666.1
##
## Number of Fisher Scoring iterations: 5
```

```r
probabilities <- predict(logit_model, newdata = test_scaled, type = "response")
roc_obj <- roc(test_scaled$label, probabilities)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```r
plot(roc_obj)
```




```r
print(auc(roc_obj))
```

```
## Area under the curve: 0.7902
```

```r
logit_model_interaction <- glm(label ~ alcohol * fix.acid, data = train_scaled, family = "binomial")
summary(logit_model_interaction)
```

```
##
## Call:
## glm(formula = label ~ alcohol * fix.acid, family = "binomial",
##     data = train_scaled)
##
## Coefficients:
```

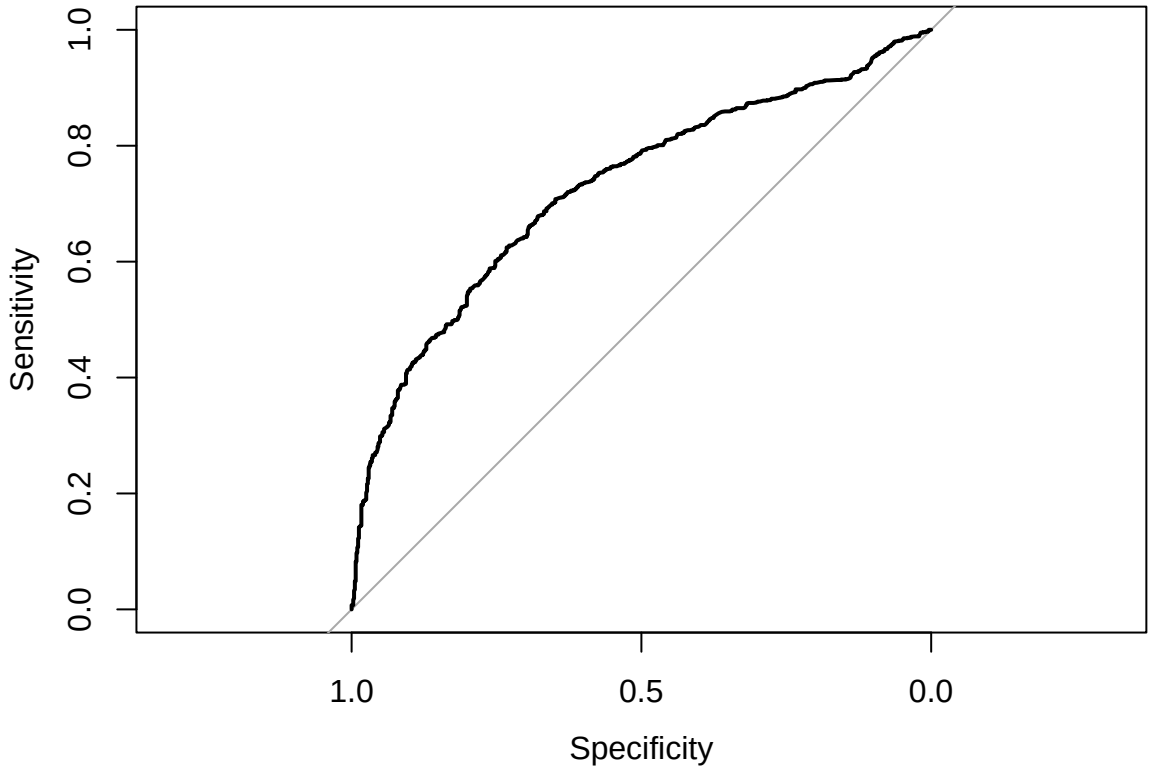
```
##                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)        0.71042    0.03625  19.599  < 2e-16 ***
## alcohol            1.05099    0.04242  24.774  < 2e-16 ***
## fix.acid          -0.12739    0.03393  -3.755 0.000173 ***
## alcohol:fix.acid -0.06127    0.04059  -1.509 0.131185
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 5979.2  on 4546  degrees of freedom
## Residual deviance: 5131.1  on 4543  degrees of freedom
## AIC: 5139.1
##
## Number of Fisher Scoring iterations: 4
```

```
probabilities <- predict(logit_model_interaction, newdata = test_scaled, type = "response")

# Generate the ROC curve
roc_obj <- roc(test_scaled$label, probabilities)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
# Plot the ROC curve
plot(roc_obj)
```




```
# Get Area Under the Curve (AUC)
print(auc(roc_obj))
```

```
## Area under the curve: 0.726
```

```
model <- glm(label ~ fix.acid + vol.acid + citric + sugar + chlorides + free.sd +
                   total.sd + density + pH + sulphates + alcohol,
                   data = train_scaled, family = "binomial")
summary(model)
```

```
##
## Call:
## glm(formula = label ~ fix.acid + vol.acid + citric + sugar +
##     chlorides + free.sd + total.sd + density + pH + sulphates +
##     alcohol, family = "binomial", data = train_scaled)
##
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  0.75205    0.03866  19.453  < 2e-16 ***
## fix.acid     0.04786    0.05006   0.956    0.339
## vol.acid    -0.79741    0.05056 -15.772  < 2e-16 ***
## citric      -0.08274    0.04400  -1.881    0.060 .
## sugar        0.30282    0.04448   6.808 9.91e-12 ***
## chlorides   -0.02537    0.04319  -0.587    0.557
```

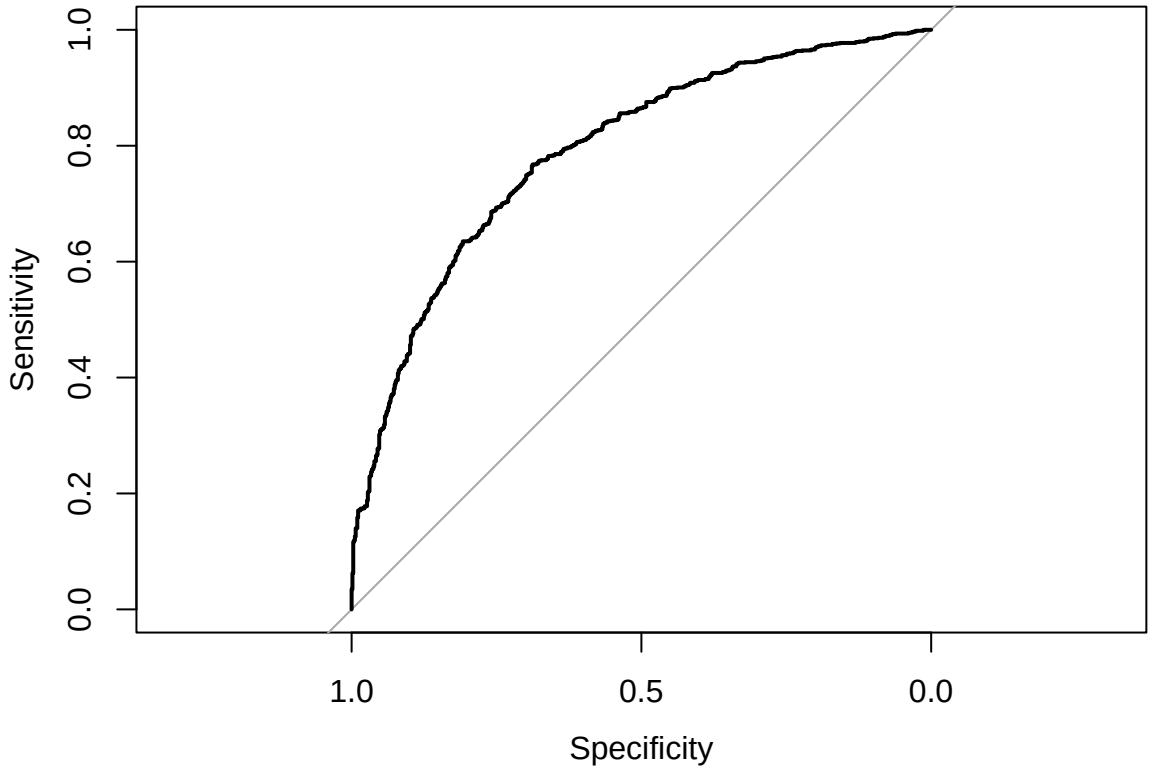
```
## free.sd      0.30628    0.05396   5.676 1.38e-08 ***
## total.sd    -0.44254    0.05917  -7.480 7.45e-14 ***
## density     -0.07060    0.05254  -1.344    0.179
## pH           0.07403    0.04367   1.695    0.090 .
## sulphates    0.26478    0.04430   5.977 2.27e-09 ***
## alcohol      1.11572    0.05188  21.505  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 5979.2  on 4546  degrees of freedom
## Residual deviance: 4642.1  on 4535  degrees of freedom
## AIC: 4666.1
##
## Number of Fisher Scoring iterations: 5
```

```
# Predict probabilities on the test set
probabilities <- predict(model, newdata = test_scaled, type = "response")

# Generate the ROC curve
roc_obj <- roc(test_scaled$label, probabilities)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
# Plot the ROC curve
plot(roc_obj)
```




```
# Get Area Under the Curve (AUC)
print(auc(roc_obj))
```

```
## Area under the curve: 0.7902
```

Based on the models I made, the one that performed the best was the one where all terms interacted with eachother. Then I computed Youden's J statisticb for the threshold.

```
library(pROC)

# Get all thresholds and statistics
coords_df <- coords(roc_obj, x = "all", ret = c("threshold", "sensitivity", "specificity"))

# Calculate Youden's J for all thresholds
youden_J <- coords_df$sensitivity + coords_df$specificity - 1

# Find the threshold that gives the maximum Youden's J
optimal_idx <- which.max(youden_J)
optimal_threshold <- coords_df$threshold[optimal_idx]
optimal_J <- youden_J[optimal_idx]
print(optimal_J)
```

```
## [1] 0.4554744
```

The calculated threshold is 0.4558348. Using that, I reclassified the predicted values from the scaled test set and generated the classification matrix. The calculated accuracy was 0.7128 and the misclassification rate is 0.2871795.

```
library(pROC)
library(caret)

predicted_class <- ifelse(probabilities > optimal_threshold, 1, 0)

cm <- confusionMatrix(as.factor(predicted_class), as.factor(test_set$label))
print(cm)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 491 287
##          1 223 949
##
##                Accuracy : 0.7385
##                  95% CI : (0.7184, 0.7579)
##     No Information Rate : 0.6338
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.447
##
##  Mcnemar's Test P-Value : 0.005276
##
##             Sensitivity : 0.6877
##             Specificity : 0.7678
##          Pos Pred Value : 0.6311
##          Neg Pred Value : 0.8097
##              Prevalence : 0.3662
##          Detection Rate : 0.2518
##    Detection Prevalence : 0.3990
##       Balanced Accuracy : 0.7277
##
##        'Positive' Class : 0
##
```

```
misclass_rate <- 1 - cm$overall['Accuracy']
print(misclass_rate)
```

```
##  Accuracy
## 0.2615385
```

I was then curious to see if other models could perform better than my logistic regression, so I tried an XGB Boost and an SVM as shown below. Neither model's accuracy is any better than my logistic regression's, even after examining the J statistic.

```
library(xgboost)
```

```
## Warning: package 'xgboost' was built under R version 4.4.3
```

```
##
## Attaching package: 'xgboost'

## The following object is masked from 'package:dplyr':
##
##     slice
```

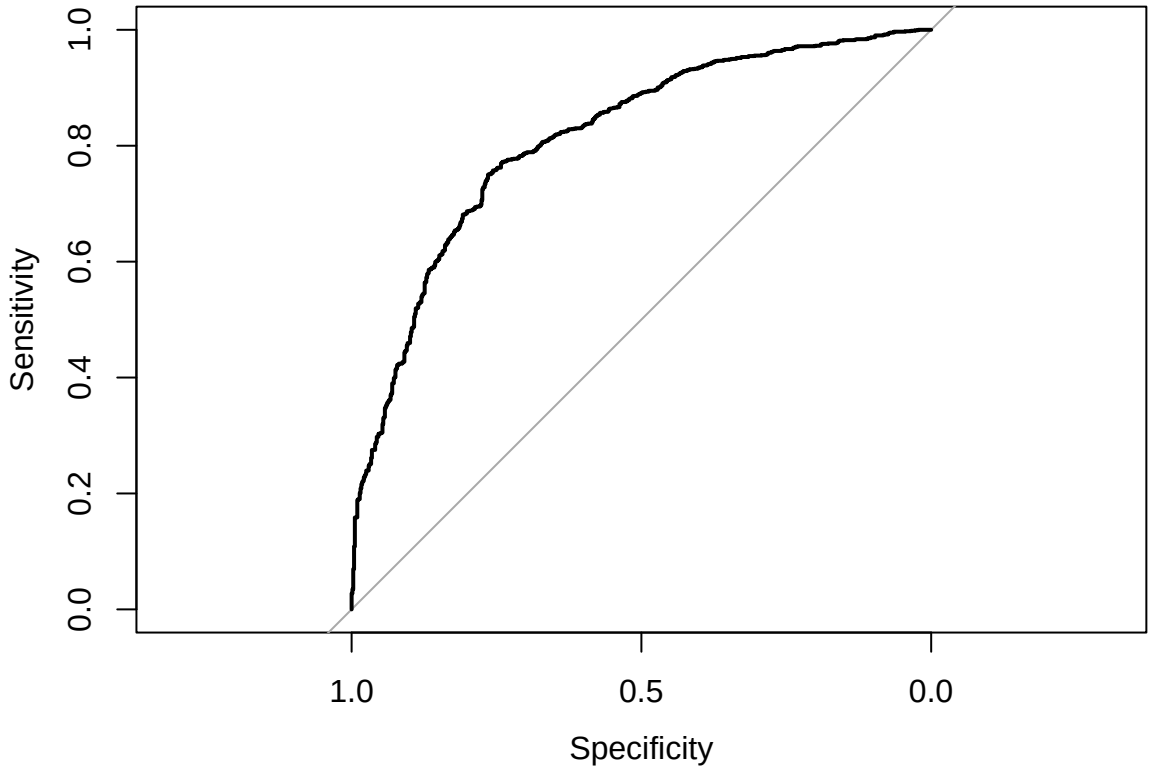
```
library(Matrix)
```

```
##
## Attaching package: 'Matrix'

## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
```

```
library(caret)
library(pROC)

train_matrix <- as.matrix(train_scaled[, predictor_cols])
test_matrix <- as.matrix(test_scaled[, predictor_cols])
label_train <- as.numeric(as.character(train_scaled$label))
label_test <- as.numeric(as.character(test_scaled$label))

xgb_model <- xgboost(data = train_matrix, label = label_train,
                     max.depth = 3, eta = 0.1, nrounds = 50,
                     objective = "binary:logistic", verbose = 0)
probabilities <- predict(xgb_model, newdata = test_matrix, type = "response")

# Generate the ROC curve
roc_obj <- roc(label_test, probabilities)
```

```
## Setting levels: control = 0, case = 1

## Setting direction: controls < cases
```

```
plot(roc_obj)
```




```
print(auc(roc_obj))
```

```
## Area under the curve: 0.8131
```

```
# Get all thresholds and statistics
coords_df <- coords(roc_obj, x = "all", ret = c("threshold", "sensitivity", "specificity"))

# Calculate Youden's J for all thresholds
youden_J <- coords_df$sensitivity + coords_df$specificity - 1

# Find the threshold that gives the maximum Youden's J
optimal_idx <- which.max(youden_J)
optimal_threshold <- coords_df$threshold[optimal_idx]
optimal_J <- youden_J[optimal_idx]
print(optimal_J)
```

```
## [1] 0.5147059
```

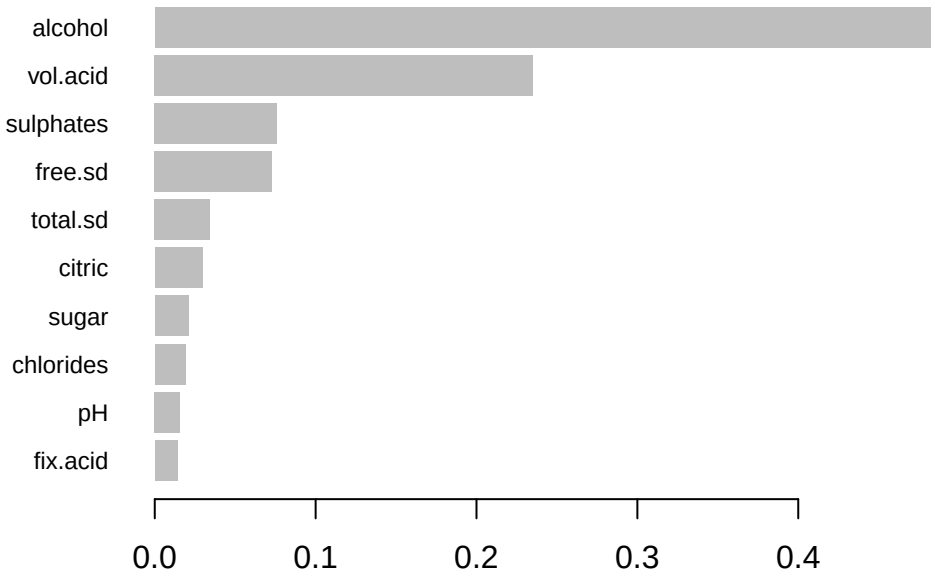
```
xgb_prob <- predict(xgb_model, test_matrix)
xgb_pred <- ifelse(xgb_prob > optimal_threshold, 1, 0)
cm_xgb <- confusionMatrix(as.factor(xgb_pred), as.factor(label_test))

print(cm_xgb)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 546 309
##          1 168 927
##
##                Accuracy : 0.7554
##                  95% CI : (0.7357, 0.7743)
##     No Information Rate : 0.6338
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.4941
##
##  Mcnemar's Test P-Value : 1.454e-10
##
##             Sensitivity : 0.7647
##             Specificity : 0.7500
##          Pos Pred Value : 0.6386
##          Neg Pred Value : 0.8466
##              Prevalence : 0.3662
##          Detection Rate : 0.2800
##    Detection Prevalence : 0.4385
##       Balanced Accuracy : 0.7574
##
##        'Positive' Class : 0
##
```

```
# Extract feature importance from the XGBoost model
importance_matrix <- xgb.importance(feature_names = colnames(train_matrix), model = xgb_model)
print(importance_matrix)
```

```
##       Feature       Gain      Cover  Frequency
##        <char>      <num>      <num>      <num>
##  1:   alcohol 0.48212684 0.30302746 0.17058824
##  2:  vol.acid 0.23507717 0.23156655 0.16764706
##  3: sulphates 0.07606911 0.06981787 0.14411765
##  4:   free.sd 0.07292585 0.08624828 0.12941176
##  5:  total.sd 0.03433898 0.10607684 0.08235294
##  6:    citric 0.02970917 0.06204010 0.07647059
##  7:     sugar 0.02085363 0.03932651 0.05882353
##  8: chlorides 0.01897751 0.03854717 0.07352941
##  9:        pH 0.01568848 0.03845512 0.05000000
## 10:  fix.acid 0.01423325 0.02489409 0.04705882
```

```
xgb.plot.importance(importance_matrix)
```




```
library(e1071)
```

```
## Warning: package 'e1071' was built under R version 4.4.3
```

```
##
## Attaching package: 'e1071'
```

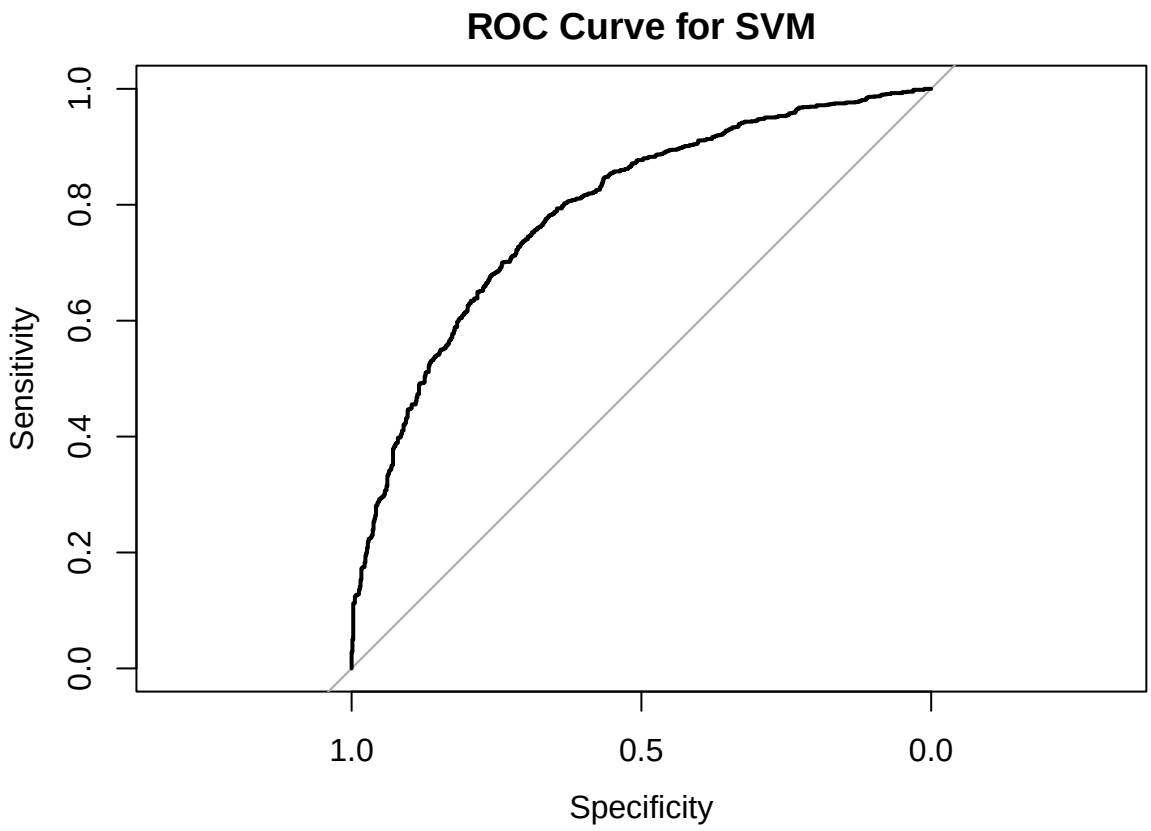
```
## The following object is masked from 'package:ggplot2':
##
##     element
```

```
library(caret)
library(pROC)

# Ensure your label is a factor
train_scaled$label <- as.factor(train_scaled$label)
test_scaled$label <- as.factor(test_scaled$label)

# Train SVM with probability estimates enabled
svm_model <- svm(label ~ fix.acid + vol.acid + citric + sugar + chlorides + free.sd +
                 total.sd + density + pH + sulphates + alcohol,
                 data = train_scaled,
                 kernel = "linear",
                 probability = TRUE)
```

```r
# Predict probabilities on test set
svm_pred_prob <- attr(predict(svm_model, test_scaled, probability = TRUE), "probabilities")[, "1"]

# ROC Curve and AUC
roc_obj <- roc(test_scaled$label, svm_pred_prob)
```

```
## Setting levels: control = 0, case = 1

## Setting direction: controls < cases
```

```r
plot(roc_obj, main = "ROC Curve for SVM")
```




```r
auc_val <-
print(auc(roc_obj))
```

```
## Area under the curve: 0.7882
```

```r
# Compute Youden's J statistic for threshold selection
coords_df <- coords(roc_obj, x = "all", ret = c("threshold", "sensitivity", "specificity"))
youden_J <- coords_df$sensitivity + coords_df$specificity - 1
optimal_idx <- which.max(youden_J)
optimal_threshold <- coords_df$threshold[optimal_idx]
optimal_J <- youden_J[optimal_idx]
print(paste("Optimal Youden's J:", optimal_J))
```

```
## [1] "Optimal Youden's J: 0.441536808898317"
```

```
print(paste("Optimal threshold:", optimal_threshold))
```

```
## [1] "Optimal threshold: 0.631036163187044"
```

```
# Class prediction using optimal threshold
svm_pred <- factor(ifelse(svm_pred_prob > optimal_threshold, "1", "0"), levels = levels(test_scaled$lab

cm_svm <- confusionMatrix(svm_pred, test_scaled$label)
print(cm_svm)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
##          0 503 325
##          1 211 911
##
##                Accuracy : 0.7251
##                  95% CI : (0.7047, 0.7448)
##     No Information Rate : 0.6338
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.4271
##
##  Mcnemar's Test P-Value : 1.056e-06
##
##             Sensitivity : 0.7045
##             Specificity : 0.7371
##          Pos Pred Value : 0.6075
##          Neg Pred Value : 0.8119
##              Prevalence : 0.3662
##          Detection Rate : 0.2579
##    Detection Prevalence : 0.4246
##       Balanced Accuracy : 0.7208
##
##        'Positive' Class : 0
##
```

Based on all the models that were made, the best by far is the logistic regression as it is the most robust. However the model's accuracy can be greatly improved by testing if other parameters and adding a term to handle noise would improve the model's accuracy.